Re-Tokenization, Segmentation, and a Blind Spot: What Content-Free Boundary Markers Do in Sequence Transformation

Anonymous authors
Paper under double-blind review

Abstract

Inserting a content-free marker (|) into the input every k characters of a character-level transformation, and asking the model to copy it through, is a cheap single-call intervention. We study what it does across seven tasks, seven open instruction-tuned models, and Gemini 3.5 Flash, using two controls that fix tokenization at one token per character: the same letter tasks with a space between letters, and exact relabelings with digits. We find three separate effects. First, on letter strings, which tokenizers merge into multi-letter tokens, markers act largely as re-tokenization: at five of the seven open models, spacing the letters with no markers at all improves accuracy more (+0.16 to +0.33) than markers do (+0.13 to +0.26), and the large advantage of the position right after a marker disappears once letters are spaced. Second, independent of tokenization, markers help strong models on long sequences: with spaced letters or digits they remove 33% to 83% of the remaining errors at the four strongest open models, with little dependence on spacing; at Qwen2.5-32B the gain persists to $N = 256$ and is the same with commas or line breaks as markers. Blocking the ten most marker-attending heads at Qwen2.5-32B removes 1% to 13% of this gain and blocking 200 removes 21% to 28%, against none for matched controls, so the gain runs partly through attention to markers, spread over many heads. Third, with a marker every two or more characters, the position right after a marker is blind to the input before it: on four dependency tasks it falls below the no-marker run in 29 of 31 task–model cells, to near chance at the stronger models, while the next position recovers. Commas and line breaks produce the same loss, and telling the model that the rule continues across the marker narrows it without removing it. In practice: separate characters before adding markers, use sparse markers on long sequences, and avoid placing a marker where an output needs the input just before it.

1 Introduction

Ask a language model to apply a simple, fully specified rule to every character of a string (shift each letter forward, substitute through a lookup table, output the XOR of each digit with the one before) and accuracy does not fade gracefully with length. It holds near ceiling on short inputs and then collapses well before any obvious capacity limit (Hou et al., 2026). Standard remedies change the computation itself: splitting the input across several calls (Zhou et al., 2023), asking for intermediate steps (Nye et al., 2021; Chen et al., 2023), or adding memory to the architecture (Dai et al., 2019; Bulatov et al., 2022; Mohtashami & Jaggi, 2023). Each costs extra calls, extra tokens, or training.

We study a cheaper intervention. A content-free marker character (|) is inserted into the input every k characters, and the model is told to copy it through to the output unchanged. The marker carries no task information, needs no extra call, and requires no training. On letter tasks it raises accuracy substantially, by up to +0.26 on average at the stronger models we test. Our question is why, and when it hurts.

A marker does more than mark a boundary. Tokenizers merge runs of letters into multi-letter tokens, and a marker after every few letters splits those runs, so on letter strings a marker is also a tokenization intervention.

We separate the two with two controls that keep one token per character whether or not markers are present: the same letter tasks with a space between letters, and exact relabelings of them with digits. We find three effects (Figure 1):

1. **On letter strings, markers largely re-tokenize** (Section 4). At all five Qwen and Gemma models, spacing the letters with no markers at all helps more than markers on unspaced letters (+0.16 to +0.33 against +0.13 to +0.26); at Mistral and Phi the two help about equally. The striking accuracy of the position right after a marker on letter tasks (positive in all 24 task–model cells, with per-model means of +0.07 to +0.34 over the no-marker run at the same positions) is largely a token-boundary effect: with spaced letters or digits that position is no better than the rest of its segment.
2. **Independent of tokenization, markers help strong models on long sequences** (Section 5). With spaced letters or digits, markers remove 33% to 83% of the remaining errors at Qwen2.5-14B and 32B, Qwen3.8-27B, and Gemma-4-31B (significant in 23 of 24 cells), and the two controls agree closely. The gain is absent at short lengths, where these models are near ceiling, appears at $N = 32$, and at $N = 64$ reaches +0.27 (spaced letters) and +0.22 (digits), with little dependence on spacing. At Qwen2.5-32B it persists to $N = 256$ (56% of remaining errors removed) and is the same with commas or line breaks as markers. At the three smallest models the gains are smaller and inconsistent (significant in 7 of 18 cells).
3. **The position after a marker is blind to the input before it** (Section 6). With a marker every two or more characters, on tasks where each output also depends on the previous input, the position right after a marker falls below the no-marker run in 29 of 31 task–model cells (every cell at the five strongest models), to near chance at the stronger models, even where the model is otherwise near ceiling. The next position recovers immediately, and rewriting the tasks so that this position needs nothing before the marker repairs it. The loss is not created by how the instruction describes the markers: telling the model they are only visual separators and that the rule continues across them narrows it but does not remove it, and commas or line breaks in place of | leave the position at chance up to $N = 256$. Pooled task-level effects on these tasks are consistent with a mix of this loss and the within-segment gains of Finding 2, which is why their sign varies with length and model.

We also test the obvious mechanism (Section 7). In Qwen2.5, some heads attend from each position to the adjacent marker, but at Qwen2.5-32B and $N = 64$, blocking the ten most marker-attending heads removes 1% of the gain on digits, 13% on spaced letters, and 19% on contiguous letters, against at most 5% for controls. Blocking 200 heads removes 21% (digits) and 28% (spaced letters), against none for controls of the same size: the gain runs partly through attention to markers, but spread thinly over many heads rather than concentrated in the few that attend to them most. Section 8 turns the findings into guidance: separate characters before adding markers, use sparse markers on long sequences, and avoid placing a marker where an output needs the input just before it.

Supporting analyses. Each finding is backed by checks reported in the appendix: re-scoring with an edit-distance alignment, which forgives dropped or inserted characters, leaves the blind spot intact (Table 8); sampling instead of greedy decoding changes neither the gain nor the blind spot (Table 18); the per-position comparisons hold model by model and by distance from the marker (Tables 6 and 7); on third-party tasks markers help a position-local substitution task and hurt a prefix-dependent algorithm (Table 10); changing the marker symbol, jittering its placement, or rephrasing the instruction changes the size of the letter-task gain at Qwen2.5-32B but not its sign, except for a reserved special token and a terse cipher-style prompt (Appendix H); and sparse markers take less wall-clock time than none, because they keep models from generating past the required length (Table 21).

2 Related work

Tokenization, formatting, and position. Subword tokenizers obscure the characters inside a token: models know only partially which characters their tokens contain (Kaushal & Mahowald, 2022), character-

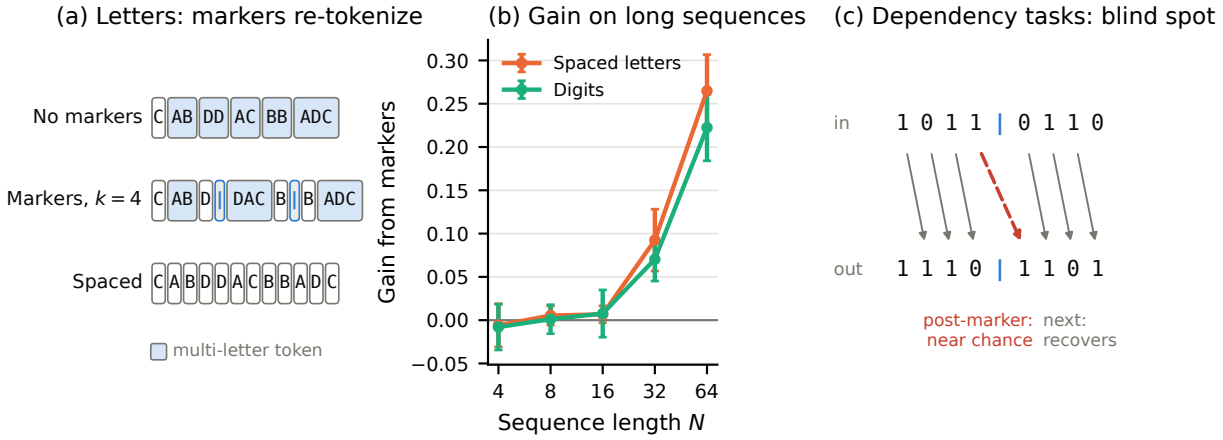


Figure 1: The three effects at a glance. (a) Qwen2.5's tokenizer merges runs of letters into multi-letter tokens; a marker every four letters splits only some of them, while spacing the letters gives one token per letter. (b) With one token per character (spaced letters or digits), markers still help the four strongest open models, from $N = 32$ on (Bounds − Base, mean over models and the three position-local tasks; whiskers are 95% intervals; same data as Figure 4a). (c) On `running_xor`, each output is the XOR of its input and the one before; the position right after a marker, whose dependency crosses the marker, falls to near chance at the stronger models, and the next position recovers.

level manipulation is a known weakness (Edman et al., 2024), and on a large benchmark of character-level tasks, longer tokens obscure where a character sits (Uzan & Pinter, 2026). The same string can be encoded many ways, and models handle non-canonical encodings surprisingly well; splitting text into characters can even improve string manipulation (Zheng et al., 2025), and retokenizing a prompt while preserving its bytes is a clean probe of segmentation sensitivity (Jain et al., 2026). Surface formatting matters more broadly: choices of separators, spacing, and casing in a prompt can change few-shot accuracy by tens of points (Sclar et al., 2024). How many tokens a text costs also varies sharply across languages and formats, with consequences for accuracy and price (Petrov et al., 2023; Ahia et al., 2023). Formatting and positional cues change arithmetic accuracy: Nogueira et al. (2021) show that the surface form of numbers matters, Shen et al. (2023) that positional description does, and Duan et al. (2023) that length generalization depends on how positions are presented. Our goal differs from these: we study a content-free marker that the model must copy through. Our tokenization controls show that, on letter strings, much of what such a marker does is the character-level segmentation these works study, and they separate it from effects that remain when segmentation is fixed.

Learned segmentation. Tokenizer-free models learn where to place boundaries. H-Net learns content-dependent chunks over bytes end to end (Hwang et al., 2026); H-Net++ adds a small cross-chunk attention module because chunking alone ignored dependencies across chunks (Zakershahrak & Ghodratnama, 2025); and MrT5 learns to delete byte tokens partway through the encoder, merging their information into the tokens that remain (Kallini et al., 2025). Our markers are the opposite case: fixed, content-free boundaries inserted at inference time into a model never trained to use them. The blind spot raises a question for learned chunkers too, namely whether the first position after a learned boundary also loses access to what came before it; the H-Net++ design choice suggests that dependencies across boundaries need explicit support there as well.

Attention sinks and separator tokens. Language models concentrate attention on a few tokens, most famously the first token, which act as sinks (Xiao et al., 2024; Gu et al., 2025; Zhang et al., 2025); Su et al. (2026) survey uses and interpretations. SepLLM (Chen et al., 2025) shows that separator tokens can summarize the segment before them, SinkLoRA (Zhang, 2024) exploits sinks during fine-tuning, and

punctuation tokens have been used as boundary anchors for trained sparse attention (Qiu et al., 2026). Oh et al. (2024) find that tabular structure in a prompt improves factual data interactions. We study untrained, content-free markers inserted into the input at inference time; our head ablations suggest that their tokenization-independent benefit does not run through the heads that attend to them most.

Attention heads. Induction heads implement in-context copying (Olsson et al., 2022), and causal head gating assigns roles to heads by intervention (Nam et al., 2025). We characterize the marker-attending heads and block them from the markers.

Long-sequence failure. Models use the middle of long contexts poorly (Liu et al., 2024), make sparse but systematic errors on flip-flop state tracking (Liu et al., 2023), and fail long-horizon tasks for reasons that compound (Wang et al., 2026; Pushkin & Abbe, 2026; Anand et al., 2026). Hou et al. (2026) quantify the accuracy cliff on repetitive deterministic tasks of the kind we use. Architectural remedies target long-sequence degradation directly; SWAN, for example, interleaves global attention layers without positional encoding with sliding-window rotary layers and extrapolates well beyond its training length (Puvvada et al., 2025). Relative position encodings such as ALiBi (Press et al., 2022) and FIRE (Li et al., 2024) likewise improve generalization to longer inputs. Markers act at the prompt, on unmodified models. The tokenization-independent benefit we find is a long-sequence effect of this kind.

Decomposition and memory. Least-to-most prompting (Zhou et al., 2023), program-of-thoughts (Chen et al., 2023), and scratchpads (Nye et al., 2021) decompose computation; Transformer-XL (Dai et al., 2019), recurrent memory (Bulatov et al., 2022), landmark tokens (Mohtashami & Jaggi, 2023), and learned context management (Kontonis et al., 2026) build segmentation or memory into the model. Content-free markers sit at the cheap end of this spectrum.

Benchmarks. Our tasks are generated fresh for every trial, following the contamination-resistant approach of BeyondBench (Srivastava et al., 2026). External checks use ProcBench (Fujisawa et al., 2024), the copying task of Jelassi et al. (2024), and CLRS-Text (Markeeva et al., 2026).

3 Setup

3.1 Tasks and formats

Every task maps an input string of length N to an output string of the same length under a deterministic rule stated fully in the prompt (Table 1). In *position-local* tasks, output y_i depends only on input x_i. In *one-step dependency* tasks, y_i also depends on x_{i-1}; in the *windowed* task, on x_{i-2}, x_{i-1}, x_i. For dependency tasks, the first position is copied as-is. Inputs are sampled uniformly with a deterministic seed per (task, length, spacing, alphabet size, trial), so every model sees the same inputs. `copy_previous` requires no computation, only a one-step lookback, so any harm from markers cannot be blamed on arithmetic; `sliding_window_max` uses a three-symbol window, so its output stays informative at every length.

The position-local tasks are posed in three formats. In the *letter* format (the default), letters are written contiguously, and tokenizers merge them: on every model we test, 48 random letters encode to 23 to 32 tokens without markers and to one token per letter with a marker after each letter. In the *spaced* format, the same letters are separated by single spaces and the model is asked to answer in the same format; 48 spaced letters encode to exactly 48 tokens on every model, with or without markers. In the *digit* format, the letters are relabeled as digits ($A \to 0$, $B \to 1$, ...), which every tokenizer we test encodes one per character. Both controls are exact transformations of the letter tasks: same seeds, same positions, same rule and prompt wording (verified in every cell). The dependency tasks use digits throughout. One caveat on `conditional_shift`: its rule depends on a symbol's position in the alphabet, which for digits equals the digit itself, so its digit version is easier.

Table 1: Tasks. A = alphabet size, M = modulus. Lengths $N \in \{4, 8, 16, 32, 64\}$. Position-local tasks are run with letters, spaced letters, and digits; dependency tasks use digits.

Task	Tier	Rule	Axis
`cyclic_shift`	position-local	next symbol in the alphabet, wrapping	$A \in \{2, 4, 8\}$
`random_permutation`	position-local	fixed arbitrary substitution (a derangement)	$A \in \{2, 4, 8\}$
`conditional_shift`	position-local	symbols at even alphabet positions forward, odd back	$A \in \{2, 4, 8\}$
`copy_previous`	one-step	$y_i = x_{i-1}$	binary
`running_xor`	one-step	$y_i = x_i \oplus x_{i-1}$	binary
`running_sum_local`	one-step	$y_i = (x_i + x_{i-1}) \bmod M$	$M \in \{2, 4, 8\}$
`sliding_window_max`	windowed	$y_i = \max(x_{i-2}, x_{i-1}, x_i)$	$A \in \{4, 8\}$

3.2 The marker intervention

In the marker condition ("Bounds"), a | is inserted after every k input characters, with $k \in \{1, 2, 4, 8, 16\}$; only $k < N$ is used. The expected output is the ground truth computed on the continuous, marker-free sequence, with markers spliced in at the same positions. The prompt states the rule, then instructs, for example: *"The input contains vertical bars (|). These are structural boundaries. DO NOT transform them. Copy them exactly to the output."* (the verb matches the task, e.g. "shift" for `cyclic_shift`). The no-marker baseline ("Base") uses the identical prompt, including that instruction; the only difference between conditions is whether markers appear in the input. Example prompts are in Appendix A. Runs also included $k = 3$, which we exclude to keep the spacing grid at powers of two.

3.3 Models and decoding

Open models are Qwen2.5-7B/14B/32B-Instruct (Qwen Team, 2024), Mistral-7B-Instruct-v0.3 (Jiang et al., 2023), Phi-3.5-mini-instruct (Abdin et al., 2024), Qwen3.8-27B (Qwen Team, 2026), and Gemma-4-31B-it (Gemma Team, 2026), run locally in bfloat16 with each model's chat template (thinking disabled for Qwen3.8 and Gemma 4) and greedy decoding. Qwen3.8-27B is a hybrid architecture: 48 of its 64 layers use Gated DeltaNet linear attention (Yang et al., 2025) and only 16 use softmax attention; Gemma-4-31B-it uses softmax attention in every layer (50 sliding-window and 10 global layers). The generation budget is $1.5N + 20$ new tokens, or $2N + 20$ at $k = 1$, where a correct marked answer is $2N - 1$ tokens. Predictions are clipped to the answer's length before scoring, so a budget can matter only by cutting an answer short; among the 86,400 generations whose raw output we stored, this happened in at most 0.6% of any condition class, at similar rates with and without markers, and in at most 0.04% at the four stronger models (Appendix B).[1] The closed model is Gemini 3.5 Flash (Google, 2026), queried through its API with the same prompts and seeds; it was run on the letter and dependency tasks but not on the two controls. Qwen2.5-32B is our primary model for head-level experiments.

3.4 Metrics and statistics

Token accuracy rate (TAR) is the fraction of output positions that match the ground truth after stripping markers and spaces from both sides and clipping the prediction to the ground-truth length. Marker placement is never scored.

Pooled effects. Only spacings $k < N$ are run, so marker runs are concentrated at longer lengths than no-marker runs, and accuracy falls with length. A pooled effect therefore compares the mean TAR of the marker runs with the no-marker mean reweighted to the same length distribution, $\Delta = \overline{\mathrm{TAR}}_{\mathrm{Bounds}} - \sum_N p_N \, \overline{\mathrm{TAR}}_{\mathrm{Base},N}$, where p_N is the share of marker runs at length N; it pools $N \leq 64$, $k \in \{1, 2, 4, 8, 16\}$, and alphabet or modulus values, and z is Δ over its standard error. Because the controls leave strong

[1] The $k = 1$ cells at $N \geq 64$ were first generated under the smaller budget, which cut them off, and then regenerated; with greedy decoding the new outputs extend the old ones, except for Mistral, Phi, and Qwen2.5-7B on two tasks, whose original runs used a different GPU model and diverge after a shared prefix with the same accuracy over it.

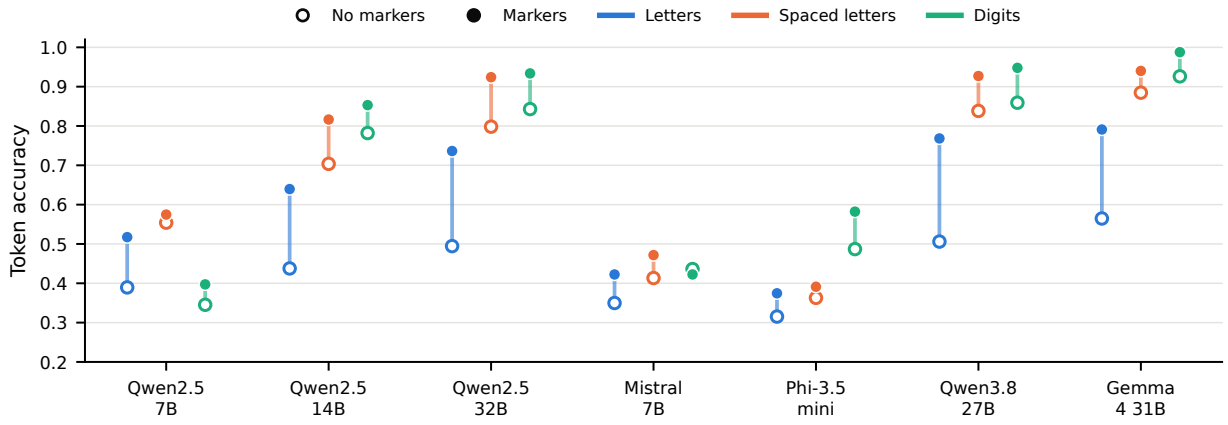


Figure 2: Token accuracy without markers (hollow) and with markers (filled) on the three position-local tasks in three formats: contiguous letters (blue), spaced letters (orange), and digits (green). Mean over tasks; $N \leq 64$, $k \in \{1, 2, 4, 8, 16\}$; no-marker values are length-matched to the marker runs (Section 3); whiskers (95% intervals) are mostly hidden by the markers. At the Qwen and Gemma models, spacing the letters without markers beats markers on contiguous letters. With tokenization fixed, markers still help the stronger models.

models near ceiling, we also report the *share of remaining errors removed*: the mean Δ over a model's tasks divided by one minus the mean length-matched no-marker accuracy. Open models use 20 to 80 trials per cell (Appendix B); Gemini uses 3. We call a pooled cell significant at $|z| \geq 3.51$, the Bonferroni-corrected two-sided $p < 0.05$ across all 112 pooled cells we test (the 70 in the overview of Section 6 and 42 for the two controls).

Per-position accuracy. For a marker run with spacing k, output position i (0-indexed, markers stripped) is *post-marker* if $i > 0$ and $i \bmod k = 0$, and *within-segment* otherwise. Each class is compared with the no-marker run's accuracy at the same position indices, which controls for accuracy varying along the sequence. Per-position results pool $N \leq 64$, $k \in \{2, 4, 8, 16\}$, and alphabet or modulus values; $k = 1$ has no within-segment positions, so per-position claims concern spacings of two or more. The unit for uncertainty is the generation, and intervals are 95%.

4 On letter strings, markers mostly re-tokenize

Figure 2 compares the three formats. On contiguous letters, markers raise accuracy at every model: by +0.26 at Qwen3.8-27B, +0.24 at Qwen2.5-32B, +0.23 at Gemma-4-31B, +0.20 at Qwen2.5-14B, +0.13 at Qwen2.5-7B, +0.07 at Mistral-7B, +0.06 at Phi-3.5-mini, and +0.22 on Gemini (all 24 cells positive, 22 significant). But at the five Qwen and Gemma models, simply spacing the letters, with no markers, helps more: +0.33 at Qwen3.8, +0.32 at Gemma 4, +0.30 at Qwen2.5-32B, +0.27 at Qwen2.5-14B, and +0.16 at Qwen2.5-7B. At these models, spaced letters without markers are more accurate than contiguous letters with markers (for example, 0.80 against 0.74 at Qwen2.5-32B and 0.89 against 0.79 at Gemma 4); at Mistral (+0.06 from spacing) and Phi (+0.05), spacing and markers help about equally. Without markers, the letter tasks are hard for these models largely because of how letters are tokenized: averaged over lengths, the four stronger models solve the digit versions at 0.82 to 0.94, against 0.47 to 0.61 with contiguous letters.

The per-position view (Figure 3) shows how. On contiguous letters, the position right after a marker beats the no-marker run at the same positions in all 24 task–model cells, with per-model means from +0.07 (Phi) to +0.34 (Qwen2.5-32B and Qwen3.8), and it beats the within-segment positions of the same marker runs in 22 of 24 cells. A marker gives the following letter its own token. With spaced letters or digits, where every symbol is already its own token, that advantage is gone: the post-marker position beats the within-segment

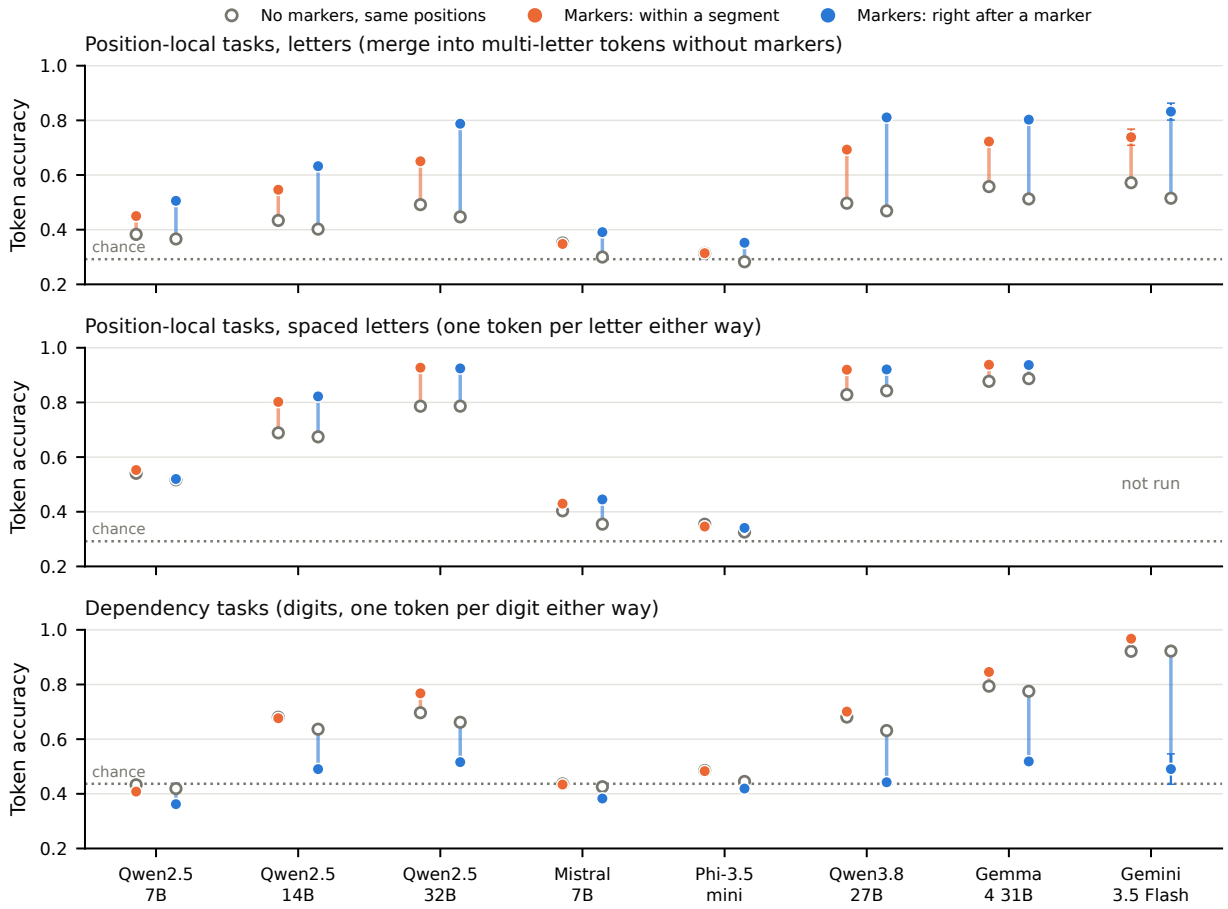


Figure 3: Per-position accuracy, every model. For each model: accuracy within a segment (orange) and at the position right after a marker (blue), each next to the no-marker run's accuracy at the same position indices (hollow gray). Mean over the tasks in each block; $N \leq 64$, $k \in \{2, 4, 8, 16\}$; whiskers are 95% intervals; dotted line: chance (always guessing the most frequent output symbol), averaged over the tasks. Top: on contiguous letters, the post-marker position gains the most. Middle: with spaced letters it is no better than the rest of its segment. Bottom: on dependency tasks it loses at every model (Section 6). Gemini was not run on spaced letters.

positions in only 10 of 21 cells in each control, and the two classes are within 0.04 of each other at every model, with the post-marker position slightly the lower of the two at Qwen2.5-7B (Appendix Table 6). The spacing profile tells the same story (Figure 4b, next section): on contiguous letters at $N = 64$, the gain is largest with a marker after every letter (+0.47 at the four stronger models) and falls steadily as markers get sparser (+0.09 at $k = 16$), as expected if markers help by splitting letter runs into shorter tokens. And the letter gain is present even on four-letter strings (+0.15), where the controls show no marker effect at all.

5 Independent of tokenization, markers help strong models on long sequences

With tokenization fixed, markers still help the stronger models. With spaced letters, the gain is +0.13 at Qwen2.5-32B, +0.11 at Qwen2.5-14B, +0.09 at Qwen3.8, and +0.06 at Gemma 4, removing 62%, 38%, 55%, and 48% of their remaining errors; with digits it is +0.06 to +0.09, removing 33% (Qwen2.5-14B) to 83% (Gemma 4). These effects are significant in all 12 spaced-letter cells and 11 of 12 digit cells at these four models. At the three smallest models the gains are smaller and inconsistent: with spaced letters, +0.02

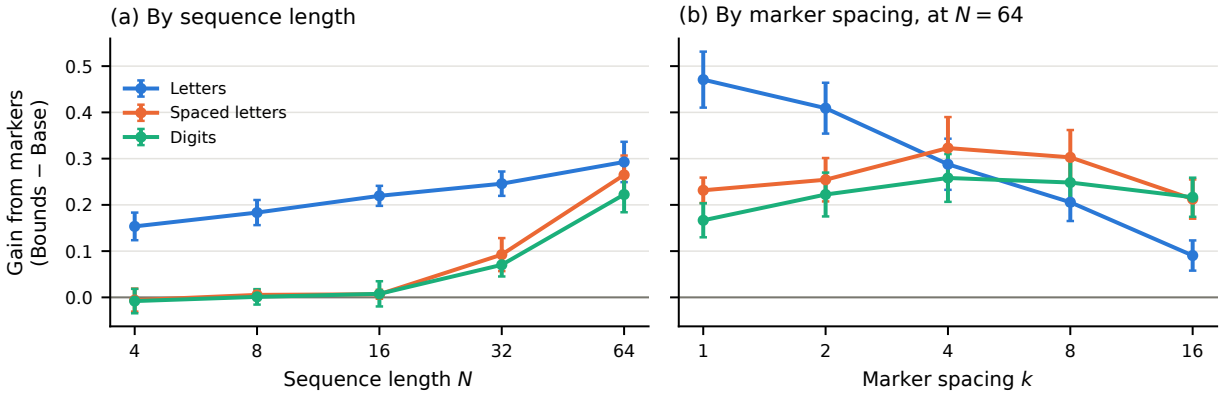


Figure 4: The marker effect (Bounds − Base) at the four stronger models (Qwen2.5-14B and 32B, Qwen3.8-27B, Gemma-4-31B), mean over the three position-local tasks. (a) By sequence length, pooled over spacing. (b) By marker spacing at $N = 64$. Whiskers are 95% intervals over the 12 model–task pairs. The two tokenization controls (spaced letters, digits) agree: no effect up to $N = 16$, a large effect at $N = 64$, and little dependence on spacing. Contiguous letters show a different signature: a gain at every length that is largest for the densest markers.

(Qwen2.5-7B), +0.06 (Mistral), and +0.03 (Phi); with digits, +0.05, −0.01, and +0.10; significant in 7 of these 18 cells.

This benefit is a long-sequence effect (Figure 4a). Up to $N = 16$, the four stronger models are near ceiling in both controls (0.92 to 0.96 without markers) and markers change nothing. At $N = 32$, accuracy without markers starts to fall and markers add +0.07 to +0.09; at $N = 64$, where accuracy without markers is 0.59 (spaced letters) and 0.68 (digits), markers add +0.27 and +0.22. In the strongest case, Qwen2.5-32B on `random_permutation` with eight digits at $N = 64$, accuracy rises from 0.41 to between 0.87 and 0.99 with any spacing from every 2 to every 16 characters. The gain depends little on spacing (Figure 4b): at $N = 64$ it is +0.21 to +0.32 for spaced letters and +0.17 to +0.26 for digits across all spacings, slightly weaker at the extremes (a marker after every character, which doubles the length of the output, or after every 16).

The gain is spread over the whole segment rather than concentrated after the marker (Figure 3, middle). A similar tokenization-independent gain appears inside segments on the dependency tasks, which use digits: within-segment positions gain +0.07 at Qwen2.5-32B, +0.05 at Gemma 4 and Gemini, and +0.02 at Qwen3.8 over the no-marker run at the same positions, though not at Qwen2.5-14B. Under edit-distance alignment these within-segment gains shrink but stay positive at the stronger models (for example, Qwen2.5-32B: +0.11 to +0.03 on `running_xor` and +0.11 to +0.05 on `running_sum_local`; Qwen3.8: +0.13 to +0.06 and +0.14 to +0.09), so part of what markers do inside segments is keep the model from slipping out of alignment. The gain also extends beyond the lengths of our main grid: at Qwen2.5-32B, on `random_permutation` with digits ($A = 8$, $k \in \{4, 16\}$), markers remove 89% to 91% of the remaining errors at $N = 64$, 78% to 81% at $N = 128$, and 56% to 57% at $N = 256$ (accuracy 0.18 without markers, 0.64 to 0.65 with), the ranges spanning `|`, commas, and line breaks as markers (Appendix Table 17). A marker every few characters appears to help a strong model keep its place in a long sequence. Section 7 shows that the heads that attend most to markers carry little of this effect, and Qwen3.8-27B, whose layers are mostly linear attention, shows it comparably to the other strong models.

6 The position after a marker is blind to the input before it

On the four dependency tasks, with a marker every two or more characters, the position right after a marker is below the no-marker run at the same positions in 29 of 31 task–model cells (Figure 3, bottom; these tasks use digits, so tokenization plays no role). The two exceptions are slight (+0.02 each, Qwen2.5-7B on `running_xor` and Phi on `sliding_window_max`); every cell at the five strongest models shows the loss. The

Table 2: Instruction wording and the blind spot: per-position change from adding markers (marker run minus the no-marker run of the same wording, at the same positions). Original: the bars are "structural boundaries". Neutral: copy each bar through. Across: the bars are only visual separators and the rule applies across them. $N \leq 64$, $k \in \{2, 4, 8, 16\}$, 20 trials per cell.

		Post-marker			Within-segment		
Task	Model	Original	Neutral	Across	Original	Neutral	Across
`running_xor`	Qwen2.5 32B	−0.19	−0.17	−0.14	+0.11	+0.06	+0.09
`running_xor`	Qwen3.8 27B	−0.06	−0.07	−0.03	+0.13	+0.07	+0.13
`running_xor`	Gemma 4 31B	−0.26	−0.25	−0.15	+0.09	+0.07	+0.03
`running_sum_local`	Qwen2.5 32B	−0.16	−0.15	−0.14	+0.11	+0.07	+0.09
`running_sum_local`	Qwen3.8 27B	−0.06	−0.07	−0.04	+0.14	+0.13	+0.14
`running_sum_local`	Gemma 4 31B	−0.41	−0.39	−0.15	+0.11	+0.08	+0.07

loss grows with model strength: −0.03 to −0.06 at Phi, Mistral, and Qwen2.5-7B; −0.15 at Qwen2.5-14B and 32B; −0.19 at Qwen3.8; −0.26 at Gemma 4; and −0.43 on Gemini, which is near ceiling without markers. At the stronger models the post-marker position is close to chance: on `running_xor` (chance 0.5) it is correct 0.51 to 0.54 of the time at Qwen2.5-14B and 32B, Qwen3.8, Gemma 4, and Gemini. The failure is confined to that position. With a marker every 16 characters, accuracy one position later is back at 0.85 (Qwen2.5-32B), 0.70 (Qwen3.8), 0.82 (Gemma 4), and 1.00 (Gemini).[2] A marker after every character ($k = 1$) is different: there every position follows a marker, and on `copy_previous` at $N = 32$ Qwen2.5-32B is more accurate with such markers (0.88) than without (0.83), plausibly because a fully regular pattern makes the lookback fixed (Section 7).

Not an artifact of scoring. Token accuracy compares characters index by index, so a dropped or inserted character misaligns everything after it. Re-scoring every run with an edit-distance alignment, which repairs such slips instead of counting the rest of the output as wrong, leaves the post-marker loss in place at every strong model and task (Appendix Table 8). At Qwen2.5-32B it is −0.24 on `running_xor` and −0.22 on `running_sum_local` under alignment, against −0.19 and −0.16 index by index.

Not a matter of wording. Our instruction calls the bars "structural boundaries", which might invite the model to treat each segment as a fresh start. We reran `running_xor` and `running_sum_local` at Qwen2.5-32B, Qwen3.8, and Gemma 4 with two other wordings (Table 2): a *neutral* one that only asks for each bar to be copied through, and an *across* one stating that the bars are only visual separators and that the digit right after a bar uses the digit right before it as its previous input. The neutral wording changes the post-marker loss by at most 0.02. The across wording narrows it, most at Gemma 4 (from −0.41 to −0.15 on `running_sum_local` and from −0.26 to −0.15 on `running_xor`) and least at Qwen2.5-32B (−0.16 to −0.14 and −0.19 to −0.14), but in every cell the post-marker position stays below the no-marker run, and at Qwen2.5-32B it remains near chance on `running_xor` (0.58; chance 0.5). The framing does not create the blind spot, and an instruction that names the dependency explicitly repairs it only in part.

Other delimiters and longer sequences. At Qwen2.5-32B we replaced `|` with a comma or a line break, each with a constraint sentence naming it, and extended `running_xor` and `running_sum_local` (modulus 4) to $N = 128$ and 256 ($k \in \{4, 16\}$, 20 trials per cell; Appendix Table 17). With every delimiter and at every length, the post-marker position is at chance: 0.48 to 0.52 on `running_xor` (chance 0.5) and 0.21 to 0.26 on `running_sum_local` (chance 0.25). Within-segment positions gain +0.16 to +0.26 and +0.09 to +0.19 over the no-marker run at the same positions. Measured against the no-marker run, the post-marker loss shrinks with length (on `running_xor`, −0.11 to −0.14 at $N = 64$, −0.03 to −0.05 at 128, and +0.03 to +0.05 at 256), but only because the no-marker run itself decays toward chance on long sequences; the post-marker position does not improve. The blind spot is a property of the boundary, not of the symbol. The line break gives the largest pooled gain in all six dependency cells, by up to 0.08.

[2]For `sliding_window_max`, the second position after a marker also needs input from before it; we count it as within-segment, which makes the dependency-task estimates for that class conservative.

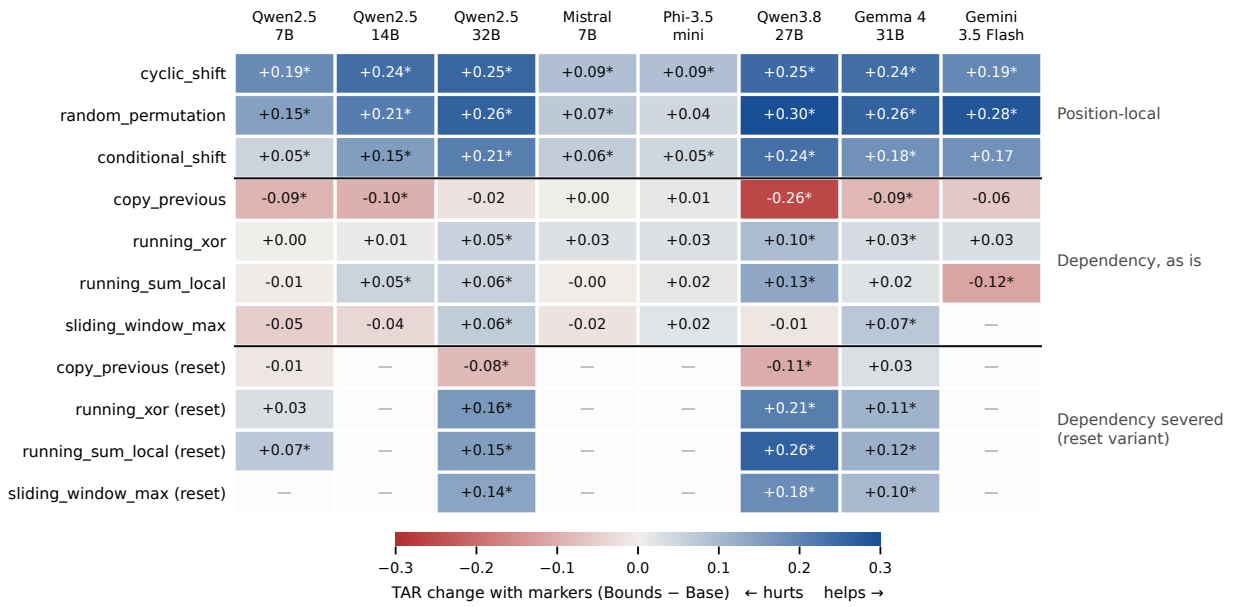


Figure 5: Pooled change in token accuracy when markers are added (Bounds − Base), contiguous-letter position-local tasks and digit dependency tasks, every model. Pooled over $N \in \{4, \ldots, 64\}$, $k \in \{1, 2, 4, 8, 16\}$, and alphabet or modulus values, with the no-marker mean length-matched to the marker runs. *: $|z| \geq 3.51$ (Bonferroni across the 112 pooled cells we test). Blue: markers help; red: markers hurt; —: not run. Bottom: dependency tasks rebuilt so the position after each marker needs nothing before it (Section 6.1).

Sampling. All runs above decode greedily. Repeating three tasks at Qwen2.5-32B with sampling ($N \in \{32, 64\}$, $k \in \{4, 16\}$; temperature 0.7 with top-p 0.8, and temperature 1.0 with top-p 0.95; one sample per prompt) changes little (Appendix Table 18). The post-marker position stays at chance, 0.51 to 0.52 on `running_xor` and 0.26 to 0.27 on `running_sum_local` under all three decoding regimes, and the gain on `random_permutation` with digits is +0.37 greedy, +0.38 at temperature 0.7, and +0.42 at temperature 1.0.

Pooled effects mix the two. Averaged over all positions (Figure 5), a dependency task's response to markers combines this post-marker loss, on about a fraction $1/k$ of positions, with the within-segment gain of Section 5 on the rest, and the pooled sign varies accordingly: of 31 dependency cells, 8 are significantly positive (all on `running_xor`, `running_sum_local`, or `sliding_window_max` at the stronger models) and 5 significantly negative (four on `copy_previous`). Averaged over tasks, the effect ranges from −0.05 (Gemini) and −0.04 (Qwen2.5-7B) to +0.04 (Qwen2.5-32B). `copy_previous`, which needs no computation, is negative at six of eight models, significantly at four, including where the model is near ceiling without markers (Qwen3.8: −0.26). The pattern with length is consistent with this mixture: short sequences admit only dense spacings ($k < N$), so a larger share of their positions sit right after a marker, while long sequences add sparse spacings and are where the within-segment gain is large. At Qwen2.5-32B, `running_xor` goes from −0.11 at $N = 4$ to +0.13 at $N = 64$, and `running_sum_local` from −0.05 to +0.14; Gemma 4's `running_sum_local` goes from −0.10 at $N = 8$ to +0.19 at $N = 64$. Qwen3.8 gains most on balance on `running_xor` and `running_sum_local` (+0.10 and +0.13): its post-marker positions fail as at every model, but its within-segment positions on those two tasks gain +0.13 and +0.14.

6.1 Redefining the post-marker position

If the post-marker position fails because it needs the input before the marker, removing that need should repair exactly that position and leave the others alone. We rebuild each dependency task as a *reset* variant: the position right after each marker is treated like the first position of the sequence, so the recurrence restarts

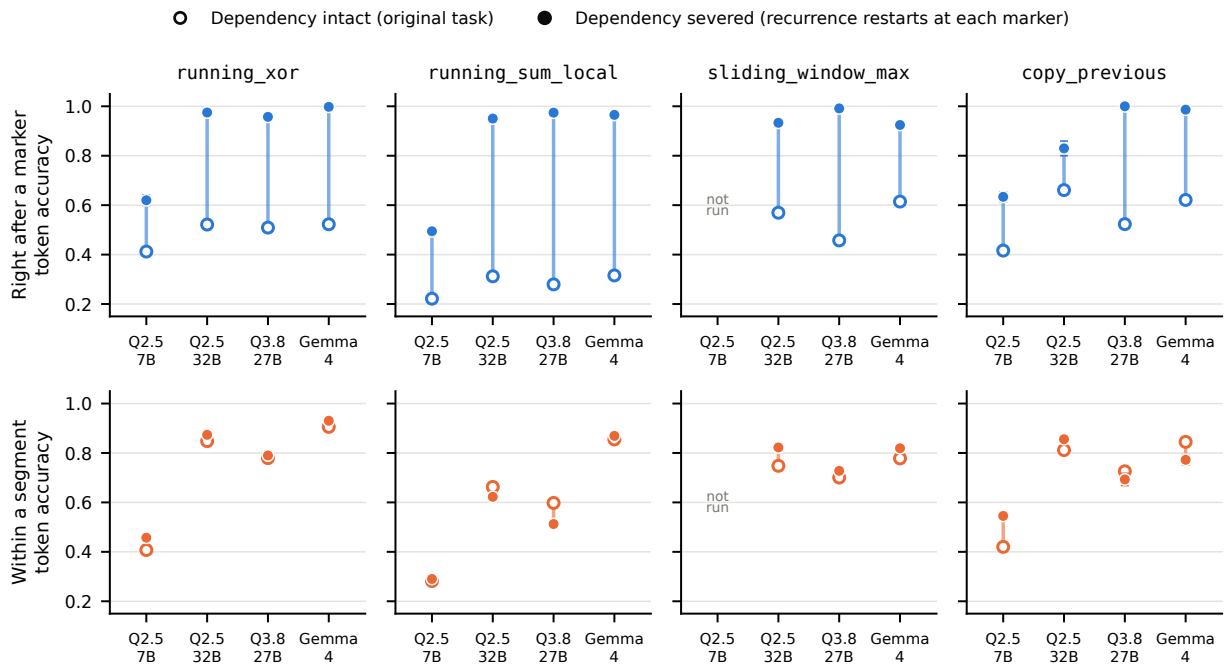


Figure 6: Reset variants, per position. Top row: accuracy at the position right after a marker, with the dependency intact (hollow) and with the recurrence restarting at each marker (filled). Bottom row: the same for within-segment positions. Models: Qwen2.5-7B and 32B, Qwen3.8-27B, Gemma-4-31B; `sliding_window_max` was not run in reset form at 7B. $N \leq 64$, $k \in \{2, 4, 8, 16\}$; whiskers are 95% intervals.

at every marker. The prompt states the new rule with a worked example (Appendix A). Reset variants were run at Qwen2.5-7B and 32B, Qwen3.8-27B, and Gemma-4-31B.

Figure 6 confirms the prediction. On `running_xor`, `running_sum_local`, and `sliding_window_max` at the three larger models, the post-marker position rises from 0.28–0.61 to 0.92–1.00, while within-segment positions change by −0.09 to +0.07. Qwen2.5-7B moves the same way with smaller gains (`running_xor` 0.41 to 0.62; `running_sum_local` 0.22 to 0.49). Two caveats bound what this shows. The reset rule makes the post-marker position easier by design (on `running_xor` it becomes "copy this digit"), so the experiment localizes the harm rather than showing a new benefit. And the reset prompt alone shifts no-marker accuracy by −0.08 to +0.06, so pooled reset effects compare different baselines (Appendix Table 9); the per-position comparison uses marker runs only.

`copy_previous` behaves differently. Its post-marker position is also repaired (0.52–0.66 to 0.83–1.00), but its within-segment positions fall at Qwen3.8 (−0.03) and Gemma 4 (−0.07), and in the original task they are below the no-marker run at seven of eight models. Because the output is a shifted copy of the input, an off-by-one slip at a marker can carry through the rest of the segment; we read this as desynchronization, a failure distinct from the post-marker loss. Edit-distance alignment supports this reading: once slips are repaired, `copy_previous`'s within-segment loss shrinks from −0.24 to −0.05 at Qwen3.8 and from −0.10 to −0.05 at Qwen2.5-14B, while its post-marker loss remains (Appendix Table 8).

6.2 External tasks

On tasks we did not design (Appendix Tables 10 and 11, Figures 10 and 11), the pattern holds. On CLRS-Text's running-minimum algorithm (Markeeva et al., 2026), where every position depends on the whole prefix, markers hurt wherever the model can do the task: −0.20 ($z = 4.5$) at Qwen2.5-32B and −0.34 ($z = 7.3$) on Gemini. On ProcBench's per-character substitution task (Fujisawa et al., 2024), a position-

Table 3: Head ablation at Qwen2.5-32B (`random_permutation`, $A = 8$, 20 trials per cell, pooled over $k \in \{2, 8\}$). "Top 10": the ten most marker-attending heads, chosen on held-out trials, are blocked from attending to markers (weights zeroed, rows renormalized). Controls: ten random heads blocked from markers, and the top ten blocked from as many random non-marker input tokens (sham). Share of gain removed is (Markers − blocked)/(Markers − No markers); "Controls" gives the larger of the two control shares.

				Markers, with heads blocked			Share of gain removed	
Format	N	No markers	Markers	Top 10	Random 10	Top 10, sham	Top 10	Controls
Letters	32	0.28	0.50	0.48	0.49	0.50	7%	$\leq 3\%$
Letters	64	0.19	0.45	0.40	0.45	0.44	19%	$\leq 3\%$
Spaced letters	32	0.70	0.99	0.98	0.98	0.98	1%	$\leq 2\%$
Spaced letters	64	0.38	0.90	0.83	0.88	0.87	13%	$\leq 5\%$
Digits	32	0.68	0.97	0.97	0.97	0.97	0%	$\leq 1\%$
Digits	64	0.41	0.96	0.95	0.97	0.96	1%	0%

local task, markers help (+0.10, $z = 6.7$ at Qwen2.5-7B; +0.20, $z = 10.8$ at 32B); its strings mix letters and digits, so part of this gain may be tokenization. On string copying (Jelassi et al., 2024), where accuracy without markers is above 0.97, the effect is null. Tagging each letter as a vowel or consonant from pretrained knowledge shows no benefit at Qwen2.5-32B (+0.01, $z = 0.5$); at 7B the baseline is at chance, so that cell is not interpretable.

7 What carries the effects? Marker-attending heads

A working hypothesis. To produce output position i, a model must find input position i (and, on dependency tasks, position $i - 1$) among many near-identical tokens. We hypothesize that markers turn this global alignment into a local one: with a marker every k characters, the model can keep its place by the markers it has passed and its offset since the last one. This would explain a gain that appears only on long sequences, is spread over the whole segment, depends little on k, and shrinks when alignment slips are forgiven in scoring. The same local frame would explain the blind spot: if "the previous input" is looked up relative to the current segment, the first position of a segment finds a marker where its predecessor should be. The experiments below test the simplest version of this account, in which a few marker-attending heads carry the effects.

In Qwen2.5, individual heads attend sharply from each position to the nearest marker (Appendix Figures 12 and 14). For the ten heads with the most attention to markers at Qwen2.5-32B, the mean percentile on a previous-token test is 95 and on an induction-head test 31 (Appendix Table 13); since heads selected for attending to an adjacent token will score high on a previous-token test, we read this only as "these are local heads". The total attention mass on markers rises with density, but so does the number of markers, so we do not claim markers are attention sinks in the sense of drawing more attention than content tokens.

Blocking those heads removes little of the gain. We block the ten most marker-attending heads from the markers during generation and compare with two controls (Table 3). On digits at Qwen2.5-32B, where markers raise accuracy from 0.41 to 0.96 at $N = 64$, blocking removes 1% of the gain (paired drop $+0.005 \pm 0.009$). At $N = 64$ it removes 13% on spaced letters ($+0.065 \pm 0.058$) and 19% on contiguous letters ($+0.049 \pm 0.030$), against at most 5% for either control; at $N = 32$ it removes 7% or less in every format. The heads that attend most to markers thus carry about none of the gain on digits, a modest and uncertain share on spaced letters, and up to a fifth on contiguous letters. An earlier ablation on contiguous letters (`cyclic_shift`, $A = 2$, $k = 2$) with heads re-identified per length removed 0% to 32% of the gain at Qwen2.5-32B and up to 74% at Qwen2.5-7B (Appendix Figure 15). At Qwen2.5-7B, marker gains in the new experiment are small and the drops from blocking are within the range of the controls (for example, $+0.03 \pm 0.05$ on spaced letters at $N = 32$, against +0.03 for random heads; Appendix Table 12). Blocking ten of the model's 2,560 heads is a narrow intervention, and the benefit may be distributed; what we can say is that it is not concentrated in the heads that attend to markers most.

Table 4: Head-count dose response at Qwen2.5-32B (`random_permutation`, $A = 8$, $N = 64$, $k = 8$, 20 trials). Share of the marker gain removed, with the paired drop in accuracy and its 95% interval. Top n: the n most marker-attending heads blocked from markers; Random n: n random heads blocked from markers; Sham: the top n blocked from as many random non-marker tokens. Markers raise accuracy from 0.38 to 0.94 (spaced letters) and from 0.42 to 0.97 (digits). The 10-head rows use $k = 8$ only, so they differ slightly from Table 3, which pools $k \in \{2, 8\}$.

Format	Heads blocked	Top n	Random n	Sham
Spaced letters	10	8% ($+0.045 \pm 0.084$)	8% ($+0.042 \pm 0.083$)	7% ($+0.039 \pm 0.083$)
	50	1% ($+0.006 \pm 0.011$)	6% ($+0.037 \pm 0.084$)	7% ($+0.041 \pm 0.083$)
	200	28% ($+0.155 \pm 0.072$)	0% (-0.002 ± 0.009)	-1% (-0.004 ± 0.011)
Digits	10	1% ($+0.003 \pm 0.007$)	-2% (-0.012 ± 0.015)	1% ($+0.003 \pm 0.004$)
	50	4% ($+0.024 \pm 0.016$)	0% ($+0.001 \pm 0.004$)	-2% (-0.009 ± 0.013)
	200	21% ($+0.115 \pm 0.083$)	-1% (-0.006 ± 0.014)	-1% (-0.007 ± 0.011)

Blocking many more heads removes part of it. To test whether the benefit is spread over heads, we repeated the ablation at $N = 64$ and $k = 8$ with the top 50 and top 200 marker-attending heads (2% and 8% of the model's heads), each with random-head and sham controls of the same size (Table 4). Blocking 50 heads removes 1% of the gain on spaced letters and 4% on digits. Blocking 200 removes 28% (paired drop $+0.155 \pm 0.072$) and 21% ($+0.115 \pm 0.083$), while 200 random heads blocked from the markers, or the top 200 blocked from as many random non-marker tokens, remove nothing (drops of -0.007 to -0.002). Because both controls of that size are null, the drop is specific to removing attention to markers rather than a general disruption. The benefit thus runs partly through attention to the markers, but thinly spread: even 200 heads carry about a quarter of it. (On spaced letters, the drops of about 0.04 for the 10-head conditions and the 50-head controls all come from one trial, the same one in each case; without it they are within 0.01 of zero.)

The same heads draw attention away from a needed input. On `copy_previous` at $N = 32$, a marker after every character keeps the marker pattern aligned with the one-step lookback, and attention from each output position to the input it needs is high (0.535). At $k = 2$, every other lookback crosses a marker; attention to the needed token falls to 0.101 and accuracy drops from 0.88 to 0.65, below the no-marker baseline of 0.83. At $k = 8$ both recover (Appendix Figure 12d). This is the attention-level counterpart of the blind spot.

A model with little softmax attention. Qwen3.8-27B, in which 48 of 64 layers use Gated DeltaNet linear attention (Yang et al., 2025) rather than softmax attention, shows the tokenization-independent gain and the blind spot comparably to the other strong models.

8 Practical considerations

Separate characters first. For letter strings, spacing the characters beats inserting markers at every Qwen and Gemma model we test, matches markers at Mistral and Phi, and costs less: 48 spaced letters are 48 tokens, against 95 with a marker after every letter. On top of spacing, markers add a further gain at the stronger models on long sequences.

Use sparse markers on long sequences. The tokenization-independent gain appears from $N \approx 32$ and depends little on spacing, so a marker every 4 to 16 characters captures it at little cost. Markers also help models stop at the right length. Without markers, on contiguous-letter tasks at $N = 64$ to 128, 18% to 95% of outputs run past the required length (Qwen2.5-32B, Qwen3.8, Gemma 4), against 0% to 13% with a marker every 8 characters. Because generation speed is constant across conditions (14 to 15 tokens/s at Qwen2.5-32B), sparse markers therefore take less time than none: 0.66 times as long at $N = 64$ and 0.44 times at $N = 128$ (Appendix Table 21). True divide-and-conquer, with one call per segment, holds accuracy

nearly flat (TAR 0.93 to 0.92 from $N = 8$ to 128) but costs about three times the wall-clock of the marker condition at $N = 64$ to 128 (Appendix Figure 19).

Avoid markers where an output needs the input before them. With a marker every two or more characters, the position after a marker fails wherever its output needs the input before the marker. Place markers only at points where nothing after them depends on what came before, or restart the computation at each marker. (A marker after every character behaves differently on `copy_previous`; see Section 6.)

What about the marker matters. Ablations on contiguous-letter `cyclic_shift` ($A = 2$, $k = 4$; Appendix Tables 14–16), where much of the effect is tokenization: replacing `|` with `;` or `#` keeps accuracy above the no-marker baseline at Qwen2.5-32B, though markers differ by up to 0.13 at $N = 128$; a reserved special token used as the marker performs far below the no-marker baseline; a frequent single-token marker beats a less frequent multi-token one with the same token budget; jittered or random placement falls below the baseline at most lengths at 7B but stays above it at 32B; and the benefit holds under formal and casual phrasings, but a terse cipher-style phrasing reverses it at 32B. These ablations cover one task and two models. Common delimiters differ from our marker in ways that may matter: whitespace between characters is our spacing control, which fixes tokenization but marks no segments; commas and semicolons, like `|`, are frequent single tokens, and `;` behaves much like `|` here; newlines also change the line structure the model sees. On the dependency tasks at Qwen2.5-32B, commas and line breaks produce the same blind spot and the same long-sequence gain as `|` (Section 6), so the advice on where to place markers applies to any delimiter. Markers also slightly reduce the share of well-formed outputs in six of seven tasks at Qwen2.5-32B (Appendix Table 20). The density that maximizes accuracy on contiguous letters falls with length (Appendix Figure 18), which we describe but do not fit (Appendix Table 19).

9 Limitations

- **Mechanism.** Blocking up to 200 marker-attending heads removes at most about a quarter of the tokenization-independent gain; we do not identify what carries the rest, and we did not test MLP or path-level interventions. Head experiments are on Qwen2.5 only and one task.
- **Controls not run everywhere.** The spaced-letter and digit controls were run on the seven open models, not on Gemini, whose tokenizer we cannot inspect.
- **Scoring.** Our main metric compares characters position by position, so a dropped or inserted character misaligns everything after it. An edit-distance-aligned re-scoring leaves the blind spot intact and attributes most of `copy_previous`'s within-segment loss to such slips (Appendix Table 8); a fully alignment-free metric would be more lenient still, especially on binary alphabets.
- **Floors and ceilings.** Raw differences in accuracy are not comparable across tasks with different baselines; the controls leave strong models near ceiling at short lengths, which is why we report shares of remaining errors. Gemini's dependency tasks start near ceiling, and some 7B, Mistral, and Phi cells start near chance.
- **Reset design.** The reset rule makes the post-marker position easier by construction, and the reset prompt shifts no-marker accuracy by up to 0.08.
- **Spacing and pooling.** Per-position claims concern spacings of two or more ($k = 1$ has no within-segment positions, and behaves differently on `copy_previous`). Pooled effects depend on how lengths are weighted; we match the no-marker baseline to the marker runs' lengths (Section 3), which raises pooled effects relative to comparing raw means (by 0.03 in the median cell and at most 0.09).
- **Models and sample size.** All open models are instruction-tuned, and five are 2024-era; we do not test base models. Reset variants at Qwen2.5-32B are single-batch (20 trials per cell); Gemini uses 3 trials per cell.

- **Length and decoding.** Core grids stop at $N = 64$; a follow-up on three tasks at Qwen2.5-32B reaches $N = 256$. Behavior over thousands of characters and at other models is untested. Main results use greedy decoding; sampling was checked on three tasks at Qwen2.5-32B only.
- **Marker variants.** Marker symbol, placement, and wording were ablated on one letter task at two models (Appendix H); commas and line breaks were tested on three tasks at Qwen2.5-32B only.
- **Position encoding and prompt format.** A marker shifts the relative offset between an output position and the input it needs, so position-encoding schemes other than those of the models we test might change the blind spot; testing this requires models trained with them. We also use each model's default chat template with no system prompt.
- **Task scope.** All tasks are synthetic character-level transformations with in-context rules, with alphabets or moduli of at most eight symbols and dependencies of one or two steps; a naturalistic task (vowel tagging) shows no benefit at 32B.

10 Conclusion

A content-free marker inserted every few characters does three separate things. On letter strings it splits multi-letter tokens, which accounts for much of its visible benefit, and spacing the characters does that job as well or better. Independently of tokenization, it helps strong models keep their place on long sequences, removing about half of their remaining errors, through attention spread thinly over many heads rather than through the few that attend to it most. And, with a marker every two or more characters, it blinds the position right after it to everything before it, which is harmless for position-local rules and damaging wherever an output needs its predecessor. Separate the characters, mark long sequences sparsely, and avoid placing markers where an output needs the input before them.

Reproducibility. [Code, prompts, raw generations, and analysis scripts: anonymized repository link.] All inputs are generated from deterministic seeds, and every prompt can be regenerated from (task, format, length, spacing, alphabet, trial).

References

Marah Abdin, Jyoti Aneja, Hany Awadalla, Ahmed Awadallah, Ammar Ahmad Awan, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Jianmin Bao, Harkirat Behl, et al. Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone. arXiv preprint arXiv:2404.14219, 2024.

Orevaoghene Ahia, Sachin Kumar, Hila Gonen, Jungo Kasai, David R. Mortensen, Noah A. Smith, and Yulia Tsvetkov. Do all languages cost the same? tokenization in the era of commercial language models. arXiv preprint arXiv:2305.13707, 2023.

Avinash Anand, Mahisha Ramesh, Avni Mittal, Ashutosh Kumar, Rishitej Reddy Vyalla, Erik Cambria, Zhengkui Wang, Timothy Liu, Aik Beng Ng, Simon See, et al. The Periodic Table of LLM Reasoning: A Structured Survey of Reasoning Paradigms, Methods, and Failure Modes. arXiv preprint arXiv:2606.11470, 2026.

Aydar Bulatov, Yuri Kuratov, and Mikhail S. Burtsev. Recurrent Memory Transformer. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2207.06881.

Guoxuan Chen, Han Shi, Jiawei Li, Yihang Gao, Xiaozhe Ren, Yimeng Chen, Xin Jiang, Zhenguo Li, Weiyang Liu, and Chao Huang. SepLLM: Accelerate Large Language Models by Compressing One Segment into One Separator. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 9008–9028, 2025.

Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. *Transactions on Machine Learning Research*, 2023. arXiv:2211.12588.